

```

read
    if ((totnoofbytes == n) || (ret == 0)) {
        return totnoofbytes;
    }
}
}

```

Similarly, while writing data, there is a function called 'mywrite'. These utility functions are in *functions.c*. The *bind* function associates a local address with the socket, and is normally used on the server side. Check out the following code:

```

serveraddress.sin_family = AF_INET;
serveraddress.sin_port = htons (MYPORT); //PORT NO
serveraddress.sin_addr.s_addr = htonl (INADDR_ANY); //IP ADDRESS
ret = bind(sd, (struct sockaddr*)&serveraddress, sizeof
(serveraddress));

```

The server's port number and IP address (and address family) that are stored in the socket's address structure are associated with the socket descriptor using the *bind* function. The purpose of *htonl (INADDR_ANY)* is to use a 'wild-card' address—if the server has multiple IP addresses, it can receive data on any of these. The *listen* function is used to change the state of the socket, so it can now *accept* connections. It also specifies the queue size of pending connections, to be retrieved.

Note the server-side accept function; the first argument is the listening *socket* (descriptor-created using the *socket* call). The function blocks till a client connects; whenever it returns successfully, a new connection socket is returned, which can be used to communicate with a particular client. Also note the second and third arguments of *accept*—the address structure that will be populated with the client's IP address and port number, and the length, which will be updated accordingly:

```

length = sizeof (cliaddr);
connfd = accept (sd, (struct sockaddr*)&cliaddr, &length);

```

In my example, the server is an 'iterative' server, which means that it can serve, at most, one client at a time. You can try to make it a concurrent server by using the *fork* or *pthread_create* calls. Also note the use of the *inet_ntop* function:

```

printf ("Data Received from %s",
    inet_ntop (AF_INET, &cliaddr.sin_addr,
    clientname, sizeof (clientname)));

```

This converts 32-bit IPv4 addresses (in network byte order) into dotted decimal notation (presentation format), and stores it in the *clientname* array.

The server code runs infinitely; to terminate it, press *Ctrl-C*. (While you are writing your own code, it's better to

write a signal handler for *Ctrl-C* and do any required cleanup; I've omitted this aspect to keep things simple.) **END** 

References

- [1] Stevens, W. Richard and Rago, Stephen A. 'Advanced Programming in the Unix Environment', 2nd Edition, Pearson Education
- [2] Stevens W. Richard, Fenner Bill, Rudoff A.M., 'Unix Network Programming' (The sockets Networking API – Vol -1), Third Edition, Pearson Education
- [3] Useful Internet links are given below:
 - a. Internet Protocol Specification: <http://www.ietf.org/rfc/rfc791.txt>
 - b. User Datagram Protocol(UDP): <http://www.ietf.org/rfc/rfc768.txt>
 - c. Transmission Control Protocol: <http://tools.ietf.org/html/rfc793>
 - d. For general programming/data structures tutorials: <http://www.mybodhizone.com/>
 - e. For programming related questions and answers: <http://www.ask.mybodhizone.com/>

Acknowledgement

I would like to thank Tanmoy Bandyopadhyay for his help in reviewing this article.

By: Swati Mukhopadhyay

The author has more than 12 years of experience in academics and corporate training. Her areas of interest are digital logic, computer architecture, computer networking, data structures, Linux, programming languages like C and C++, shell scripting and Python scripting. For any queries or feedback, she can be reached at swati.mukerjee@gmail.com.



Established: 1998 | Registered Users: 250,000+

Monthly Statistics

Page Impressions: 750,000+

Unique Techies: 150,000



electronicsforu.com BETA

If it's electronics, it's here

For any queries, please contact our team at efyeng@efyindia.com OR +91-11-26810601